static_vector

> A dynamically-resizable vector with fixed capacity and embedded storage (revision 3)

Document number: P0843r3.

Date: 2019-01-20.

Project: Programming Language C++, Library Working Group.

Audience: LEWG.

Reply-to: Gonzalo Brito Gadeschi <gonzalo.gadeschi at rwth-aachen dot de>.

Table of contents

Changelog

Revision 3

- Include LWG design questions for LEWG.
- Incorporates LWG feedback.

Revision 2

- Replace the placeholder name `fixed_capacity_vector` with `static_vector`
- Remove `at` checked element access member function.
- Add changelog section.

Revision 1

- Minor style changes and bugfixes.

Design Questions from LWG to LEWG

LWG asks LEWG to re-consider the following two design decisions:

- In this document, exceeding the capacity in methods like `static_vector::push_back` is a pre-condition violation, that is, if the capacity is exceeded, the behavior is undefined. LWG suggested that exceeding the capacity in these methods should `std::abort` instead. The trade-offs in this space are discussed in Section 4.4 Exception safety of this proposal.

- In this document, `<static_vector>` is a *free-standing* header, this is now clarified in Section 5. Technical Specification. LWG suggests that `static_vector` should be included in `<vector>` instead.

1. Introduction

This paper proposes a modernized version of `boost::container::static_vector<T,Capacity>` [1]. That is, a dynamically-resizable `vector` with compile-time fixed capacity and contiguous embedded storage in which the elements are stored within the vector object itself.

Its API closely resembles that of `std::vector<T, A>`. It is a contiguous container with `O(1)` insertion and removal of elements at the end (non-amortized) and worst case `O(size())` insertion and removal otherwise. Like `std::vector`, the elements are initialized on insertion and destroyed on removal. For trivial `value_type`s, the vector is fully usable inside `constexpr` functions.

2. Motivation and Scope

The `static_vector` container is useful when:

- memory allocation is not possible, e.g., embedded environments without a free store, where only a stack and the static memory segment are available,
- memory allocation imposes an unacceptable performance penalty, e.g., with respect to latency,
- allocation of objects with complex lifetimes in the *static*-memory segment is required,
- `std::array` is not an option, e.g., if non-default constructible objects must be stored,
- a dynamically-resizable array is required within `constexpr` functions,
- the storage location of the `static_vector` elements is required to be within the `static_vector` object itself (e.g. to support `memcopy` for serialization purposes).

3. Existing practice

There are at least 3 widely used implementations of `static_vector`: Boost.Container [1], EASTL [2], and Folly [3]. The main difference between these is that `Boost.Container` implements `static_vector` as a standalone type with its own guarantees, while both EASTL and Folly implement it by adding an extra template parameter to their `small_vector` types.

A `static_vector` can also be poorly emulated by using a custom allocator, like for example Howard Hinnant's `stack_alloc` [4], on top of `std::vector`.

This proposal shares a similar purpose with P0494R0 [5] and P0597R0: `std::constexpr_vector<T>` [6]. The main difference is that this proposal closely follows `boost::container::static_vector` [1] and proposes to standardize existing practice. A prototype implementation of this proposal for standardization purposes is provided here: `http://github.com/gnzlbg/fixed_capacity_vector`.

4. Design Decisions

The most fundamental question that must be answered is:

> Should `static_vector` be a standalone type or a special case of some other type?

The EASTL [2] and Folly [3] special case `small_vector`, e.g., using a 4th template parameter, to make it become a `static_vector`. The paper P0639R0: Changing attack vector of the `constexpr_vector` [7] proposes improving the `Allocator` concepts to allow `static_vector`, among others, to be implemented as a special case of `std::vector` with a custom allocator.

Both approaches run into the same fundamental issue: `static_vector` methods are identically-named to those of `std::vector` yet they have subtly different effects, exception-safety, iterator invalidation, and complexity guarantees.

This proposal follows `boost::container::static_vector<T,Capacity>` [1] closely and specifies the semantics that `static_vector` ought to have as a standalone type. As a library component this delivers immediate value.

I hope that having the concise semantics of this type specified will also be helpful for those that want to generalize the `Allocator` interface to allow implementing `static_vector` as a `std::vector` with a custom allocator.

4.1 Storage/Memory Layout

The container models `ContiguousContainer`. The elements of the `static_vector` are contiguously stored and properly aligned within the `static_vector` object itself. The exact location of the contiguous elements within the `static_vector` is not specified. If the `Capacity` is zero the container has zero size:

```
static_assert(is_empty_v<static_vector<T, 0>>); // for all T
```

This optimization is easily implementable, enables the EBO, and felt right.

4.2 Move semantics

The move semantics of `static_vector<T, Capacity>` are equal to those of `std::array<T, Size>`. That is, after

```
static_vector a(10);
static_vector b(std::move(a));
```

the elements of `a` have been moved element-wise into `b`, the elements of `a` are left in an initialized but unspecified state (have been moved from state), the size of `a` is not altered, and `a.size() == b.size()`.

Note: this behavior differs from `std::vector<T, Allocator>`, in particular for the similar case in which `std::allocator_traits<Allocator>::propagate_on_container_move_assignment` is `false`. In this situation the state of `std::vector` is initialized but unspecified.

4.3 `constexpr` support

The API of `static_vector<T, Capacity>` is `constexpr`. If `is_trivially_copyable_v<T> && is_default_constructible_v<T>` is `true`, `static_vector`s can be seamlessly used from `constexpr` code. This allows using `static_vector` as a `constexpr_vector` to, e.g., implement other constexpr containers.

The implementation cost of this is small: the prototye implementation specializes the storage for trivial types to use a C array with value-initialized elements and a defaulted destructor.

This changes the algorithmic complexity of `static_vector` constructors for trivial-types from "Linear in `N`" to "Constant in `Capacity`". When the value-initialization takes place at run-time, this difference in behavior might be signficiant: `static_vector<non_trivial_type, 38721943228473>(4)` will only initialize 4 elements but `static_vector<trivial_type, 38721943228473>(4)` must value-initialize the `38721943228473 - 4` excess elements to be a valid `constexpr` constructor.

Very large `static_vector`'s are not the target use case of this container class and will have, in general, worse performance than, e.g., `std::vector` (e.g. due to moves being `O(N)`).

Future improvements to `constexpr` (e.g. being able to properly use `std::aligned_storage` in constexpr contexts) allow improving the performance of `static_vector` in a backwards compatible way.

4.4 Exception Safety

The only operations that can actually fail within `static_vector<value_type, Capacity>` are:

1. `value_type`'s constructors/assignment/destructors/swap can potentially throw,
2. Mutating operations exceeding the capacity (`push_back`, `insert`, `emplace`, `static_vector(value_type, size)`, `static_vector(begin, end)`...).
3. Out-of-bounds unchecked access:
    - 3.1 `front` / `back` / `pop_back` when empty, `operator[]` (unchecked random-access).

When `value_type`'s operations are invoked, the exception safety guarantees of `static_vector` depend on whether these operations can throw. This is detected with `noexcept`.

Since its `Capacity` is fixed at compile-time, `static_vector` never dynamically allocates memory, the answer to the following question determines the exception safety for all other operations:

> What should `static_vector` do when its `Capacity` is exceeded?

Three main answers were explored in the prototype implementation:

1. Throw an exception.
2. Abort the process.
3. Make this a precondition violation.

Throwing an exception is appealing because it makes the interface slightly more similar to that of `std::vector`. However, which exception should be thrown? It cannot be `std::bad_alloc`, because nothing is being allocated. It could throw either `std::out_of_bounds` or `std::logic_error` but in any case the interface does not end up being equal to that of `std::vector`.

Aborting the process avoids the perils of undefined behavior but comes at the cost of enforcing a particular "error handling" mechanism in the implementation, which would not allow extending it to use, e.g., Contracts, in the future.

The alternative is to make not exceeding the capacity a precondition on the `static_vector`'s methods. This approach allows implementations to provide good run-time diagnostics if they so desire, e.g., on debug builds by means of an assertion, and makes implementation that avoid run-time checks conforming as well. Since the mutating methods have a precondition, they have narrow contracts, and are not conditionally `noexcept`. This provides implementations that desire throwing an exception the freedom to do so, and it also provides the standard the freedom to improve these APIs by using contracts in the future.

This proposal previously chooses this path and makes exceeding the `static_vector`'s capacity a precondition violation that results in undefined behavior. Throwing `checked_xxx` methods can be provided in a backwards compatible way.

4.5 Iterator invalidation

The iterator invalidation rules are different than those for `std::vector`, since:

- moving a `static_vector` invalidates all iterators,
- swapping two `static_vector`s invalidates all iterators, and
- inserting elements at the end of an `static_vector` never invalidates iterators.

The following functions can potentially invalidate the iterators of `static_vector`s: `resize(n)`, `resize(n, v)`, `pop_back`, `erase`, and `swap`.

4.6 Naming

The `static_vector` name was chosen after considering the following names via a poll in LEWG:

- `array_vector`: a vector whose storage is backed up by a raw array.
- `bounded_vector`: clearly indicates that the the size of the vector is bounded.
- `fixed_capacity_vector`: clearly indicates that the capacity is fixed.
- `static_capacity_vector`: clearly indicates that the capacity is fixed at compile time (static is overloaded).
- `static_vector` (Boost.Container): due to "static" / compile-time allocation of the elements. The term `static` is, however, overloaded in C++ (e.g. `static` memory?).
- `embedded_vector<T, Capacity>`: since the elements are "embedded" within the `fixed_capacity_vector` object itself. Sadly, the name `embedded` is overloaded, e.g., embedded systems.
- `inline_vector`: the elements are stored "inline" within the `fixed_capacity_vector` object itself. The term `inline` is, however, already overloaded in C++ (e.g. `inline` functions => ODR, inlining, `inline` variables).
- `stack_vector`: to denote that the elements can be stored on the stack. Is confusing since the elements can be on the stack, the heap, or the static memory segment. It also has a resemblance with `std::stack`.
- `limited_vector`
- `vector_n`

The names `static_vector` and `vector_n` tied in the number of votes. Many users are already familiar with the most widely implementation of this container (`boost::container::static_vector`), which gives `static_vector` an edge over a completely new name.

4.7 Future extensions

The following extensions could be added in a backwards compatible way:

- utilities for hiding the concrete type of vector-like containers (e.g. `any_vector_ref<T>` / `any_vector<T>`).
- default-initialization of the vector elements (as opposed to value-initialization): e.g. by using a tagged constructor with a `default_initialized_t` tag.
- tagged-constructor of the form `static_vector(with_size_t, std::size_t N, T const& t = T())` to avoid the complexity introduced by initializer lists and braced initialization:

```
using vec_t = static_vector<std::size_t, Capacity>;
vec_t v0(2);  // two-elements: 0, 0
vec_t v1{2};  // one-element: 2
vec_t v2(2, 1);  // two-elements: 1, 1
vec_t v3{2, 1};  // two-elements: 2, 1
```

All these extensions are generally useful and not part of this proposal.

5. Technical specification

Note to editor: This enhancement is a pure header-only addition to the C++ standard library as the *freestanding* `<static_vector>` header. It belongs in the "Sequence containers" (`\ref{sequences}`) part of the "Containers library" (`\ref{containers}`) as "Class template `static_vector`".

Note to LWG: one of the primary use cases for this container is embedded/freestanding. An alternative to adding a new `<static_vector>` header would be to add `static_vector` to any of the *freestanding* headers. None of the current *freestanding* headers is a good semantic fit.

---

5. Class template `static_vector`

Changes to `library.requirements.organization.headers` table "C++ library headers": add `<static_vector>`.

Changes to `library.requirements.organization.compliance` table "C++ headers for freestanding implementations": add row:

> [static_vector] Static vector `<static_vector>`

Changes to `container.requirements.general`.

The note of Table "Container Requirements" should be changed to contain `static_vector` as well:

> Those entries marked “(Note A)” or “(Note B)” have linear complexity for `array` and `static_vector` and have constant complexity for all other standard containers. [Note: The algorithm equal() is defined in [algorithms]. — end note]

5.1 Class template `static_vector` overview

- i. A `static_vector` is a contiguous container that supports constant time insert and erase operations at the end; insert and erase in the middle take linear time. Its capacity is part of its type and its elements are stored within the `static_vector` object itself, meaning that that if `v` is a `static_vector<T, N>` then it obeys the identity `&v[n] == &v[0] + n` for all `0 <= n <= v.size()`.
- ii. A `static_vector` satisfies the container requirements (`\ref{container.requirements}`) with the exception of the `swap` member function, whose complexity is linear instead of constant. It satisfies the sequence container requirements, including the optional sequence container requirements (`\ref{sequence.reqmts}`), with the exception of the `push_front`, `pop_front`, and `emplace_front` member functions, which are not provided. It satisfies the reversible container (`\ref{container.requirements}`) and contiguous container (`\ref{container.requirements.general}`) requirements. Descriptions are provided here only for operations on `static_vector` that are not described in one of these tables or for operations where there is additional semantic information.
- iii. Class `static_vector` relies on the implicitly-declared special member functions (`\ref{class.default.ctor}`, `\ref{class.dtor}`, and `\ref{class.copy.ctor}`) to conform to the container requirements table in `\ref{container.requirements}`. In addition to the requirements specified in the container requirements table, the move constructor and move assignment operator for array require that `T` be `Cpp17MoveConstructible` or `Cpp17MoveAssignable`, respectively.

```
namespace std {

template <typename T, size_t N>
class static_vector {
public:
// types:
using value_type = T;
using pointer = T*;
using const_pointer = const T*;
using reference = value_type&;
using const_reference = const value_type&;
using size_type =  size_t;
using difference_type = ptrdiff_t;
using iterator = implementation-defined;  // see [container.requirements]
using const_iterator = implementation-defined; // see [container.requirements]
using reverse_iterator = std::reverse_iterator<iterator>;
using const_reverse_iterator = std::reverse_iterator<const_iterator>;

// 5.2, copy/move construction:
constexpr static_vector() noexcept;
constexpr static_vector(const static_vector&);
constexpr static_vector(static_vector&&);
constexpr explicit static_vector(size_type n);
constexpr static_vector(size_type n, const value_type& value);
template <class InputIterator>
constexpr static_vector(InputIterator first, InputIterator last);
constexpr static_vector(const static_vector& other);
constexpr static_vector(static_vector&& other);
```

```
constexpr static_vector(initializer_list<value_type> il);

// 5.3, copy/move assignment:
constexpr static_vector& operator=(const static_vector& other)
  noexcept(is_nothrow_copy_assignable_v<value_type>);
constexpr static_vector& operator=(static_vector&& other);
  noexcept(is_nothrow_move_assignable_v<value_type>);
template <class InputIterator>
constexpr void assign(InputIterator first, InputIterator last);
constexpr void assign(size_type n, const value_type& u);
constexpr void assign(initializer_list<value_type> il);

// 5.4, destruction
~static_vector();

// iterators
constexpr iterator               begin()         noexcept;
constexpr const_iterator         begin()   const noexcept;
constexpr iterator               end()           noexcept;
constexpr const_iterator         end()     const noexcept;
constexpr reverse_iterator       rbegin()        noexcept;
constexpr const_reverse_iterator rbegin()  const noexcept;
constexpr reverse_iterator       rend()          noexcept;
constexpr const_reverse_iterator rend()    const noexcept;
constexpr const_iterator         cbegin()        noexcept;
constexpr const_iterator         cend()    const noexcept;
constexpr const_reverse_iterator crbegin()       noexcept;
constexpr const_reverse_iterator crend()   const noexcept;

// 5.5, size/capacity:
constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
static constexpr size_type max_size() noexcept;
static constexpr size_type capacity() noexcept;
constexpr void resize(size_type sz);
constexpr void resize(size_type sz, const value_type& c);

// 5.6, element and data access:
constexpr reference       operator[](size_type n);
constexpr const_reference operator[](size_type n) const;
constexpr reference       front();
constexpr const_reference front() const;
constexpr reference       back();
constexpr const_reference back() const;
constexpr       T* data()       noexcept;
constexpr const T* data() const noexcept;

// 5.7, modifiers:
constexpr iterator insert(const_iterator position, const value_type& x);
constexpr iterator insert(const_iterator position, value_type&& x);
constexpr iterator insert(const_iterator position, size_type n, const value_type& x);
template <class InputIterator>
  constexpr iterator insert(const_iterator position, InputIterator first, InputIterator last);
constexpr iterator insert(const_iterator position, initializer_list<value_type> il);

template <class... Args>
  constexpr iterator emplace(const_iterator position, Args&&... args);
template <class... Args>
  constexpr reference emplace_back(Args&&... args);
constexpr void push_back(const value_type& x);
constexpr void push_back(value_type&& x);

constexpr void pop_back();
constexpr iterator erase(const_iterator position);
constexpr iterator erase(const_iterator first, const_iterator last);

constexpr void clear() noexcept;
```

```
constexpr void swap(static_vector& x)
  noexcept(is_nothrow_swappable_v<value_type> &&
           is_nothrow_move_constructible_v<value_type>);
};

template <typename T, size_t N>
constexpr bool operator==(const static_vector<T, N>& a, const static_vector<T, N>& b);
template <typename T, size_t N>
constexpr bool operator!=(const static_vector<T, N>& a, const static_vector<T, N>& b);
template <typename T, size_t N>
constexpr bool operator<(const static_vector<T, N>& a, const static_vector<T, N>& b);
template <typename T, size_t N>
constexpr bool operator<=(const static_vector<T, N>& a, const static_vector<T, N>& b);
template <typename T, size_t N>
constexpr bool operator>(const static_vector<T, N>& a, const static_vector<T, N>& b);
template <typename T, size_t N>
constexpr bool operator>=(const static_vector<T, N>& a, const static_vector<T, N>& b);

// 5.8, specialized algorithms:
template <typename T, size_t N>
constexpr void swap(static_vector<T, N>& x, static_vector<T, N>& y)
  noexcept(noexcept(x.swap(y)));

}  // namespace std
```

5.2 `static_vector` constructors

```
constexpr static_vector() noexcept;
```

> - *Effects*: Constructs an empty `static_vector`.
> - *Ensures*: `empty()`.
> - *Complexity*: Constant.

```
constexpr static_vector(static_vector&& rv);
```

> - *Effects*: Constructs a `static_vector` by move-inserting the elements of `rv`.
> - *Requires*: `std::is_move_constructivle<value_type>`.
> - *Ensures*: The `static_vector` is equal to the value that `rv` had before this construction.
> - *Complexity*: Linear in `rv.size()`.

```
constexpr explicit static_vector(size_type n);
```

> - *Effects*: Constructs a `static_vector` with `n` default-inserted elements.
> - *Requires*: `std::is_default_constructible<value_type>`.
> - *Expects*: `n <= capacity()`.
> - *Complexity*: Linear in `n`.

```
constexpr static_vector(size_type n, const value_type& value);
```

- *Effects*: Constructs a `static_vector` with `n` copies of `value`.
- *Requires*: `std::is_copy_constructible<value_type>`
- *Expects*: `n <= capacity()`.
- *Complexity*: Linear in `n`.

```
template <class InputIterator>
constexpr static_vector(InputIterator first, InputIterator last);
```

- *Effects*: Constructs a `static_vector` equal to the range `[first, last)`
- *Requires*: `std::is_constructible<value_type, decltype(*first)>`
- *Expects*: `distance(first, last) <= capacity()`
- *Complexity*: Linear in `distance(first, last)`.

5.3 Destruction

```
~static_vector();
```

Effects: Destroys the `static_vector` and its elements.

Requires: This destructor is trivial if the destructor of `value_type` is trivial.

5.4 Size and capacity

```
static constexpr size_type capacity() noexcept
static constexpr size_type max_size() noexcept
```

- *Returns*: `N`.

```
constexpr void resize(size_type sz);
```

- *Effects*: If `sz < size()`, erases the last `size() - sz` elements from the sequence. Otherwise, appends `sz - size()` default-constructed elements.
- *Requires*: `std::is_default_constructible<value_type>`.
- *Expects*: `sz <= capacity()`.
- *Complexity*: Linear in `sz`.

```
constexpr void resize(size_type sz, const value_type& c);
```

- *Effects*: If `sz < size()`, erases the last `size() - sz` elements from the sequence. Otherwise, appends `sz - size()` copies of `c`.
- *Requires*: `std::is_copy_constructible<value_type>`.
- *Expects*: `sz <= capacity()`.
- *Complexity*: Linear in `sz`.

5.5 Element and data access

```
constexpr       T* data()       noexcept;
constexpr const T* data() const noexcept;
```

- *Returns*: A pointer such that `[data(), data() + size())` is a valid range. For a non-empty `static_vector`, `data() == addressof(front())`.
- *Complexity*: Constant time.

5.6 Modifiers

Note to LWG: All modifiers have a pre-condition on not exceeding the `capacity()` when inserting elements. That is, exceeding the `capacity()` of the vector is undefined behavior. This supports some of the major use cases of this container (embedded, freestanding, etc.) and was required by the stakeholders during LEWG review. Currently, this provides maximum freedom to the implementation to choose an appropriate behavior: `abort`, `assert`, throw an exception (which exception? `bad_alloc`? `logic_error`? `out_of_bounds`? etc.). In the future, this freedom allows us to specify these pre-conditions using contracts.

Note to LWG: Because all modifiers have preconditions, they all have narrow contracts and are not unconditionally `noexcept`.

```
constexpr iterator insert(const_iterator position, const value_type& x);
```

- *Effects*: Inserts `x` at `position` and invalidates all references to elements after `position`.
- *Expects*: `size() < capacity()`.
- *Requires*: `std::is_copy_constructible<value_type>`.
- *Complexity*: Linear in `size()`.
- *Remarks*: If an exception is thrown by `value_type`'s copy constructor and `is_nothrow_move_constructible_v<value_type>` is `true` there are no effects. Otherwise, if an exception is thrown by `value_type`'s copy constructor the effects are *unspecified*.

```
constexpr iterator insert(const_iterator position, size_type n, const value_type& x);
```

- *Effects*: Inserts `n` copies of `x` at `position` and invalidates all references to elements after `position`.
- *Expects*: `n <= capacity() - size()`.
- *Requires*: `std::is_copy_constructible<value_type>`.
- *Complexity*: Linear in `size()` and `n`.

- *Remarks*: If an exception is thrown by `value_type`'s copy constructor and `is_nothrow_move_constructible_v<value_type>` is `true` there are no effects. Otherwise, if an exception is thrown by `value_type`'s copy constructor the effects are *unspecified*.

```
constexpr iterator insert(const_iterator position, value_type&& x);
```

- *Effects*: Inserts `x` at `position` and invalidates all references to elements after `position`.
- *Expects*: `size() < capacity()`.
- *Requires*: `std::is_move_constructible<value_type>`.
- *Complexity*: Linear in `size()`.
- *Remarks*: If an exception is thrown by `value_type`'s move constructor the effects are *unspecified*.

```
template <typename InputIterator>
constexpr iterator insert(const_iterator position, InputIterator first, InputIterator last);
```

- *Effects*: Inserts elements in range `[first, last)` at `position` and invalidates all references to elements after `position`.
- *Expects*: `distance(first, last) <= capacity() - size()`.
- *Requires*: `std::is_constructible<value_type, decltype(*first)>`.
- *Complexity*: Linear in `size()` and `distance(first, last)`.
- *Remarks*: If an exception is thrown by `value_type` constructor from `decltype(*first)` and `is_nothrow_move_constructible_v<value_type>` is `true` there are no effects. Otherwise, if an exception is thrown by `value_type`'s constructor from `decltype(*first)` the effects are *unspecified*.

```
constexpr iterator insert(const_iterator position, initializer_list<value_type> il);
```

- *Effects*: Inserts elements of `il` at `position` and invalidates all references to elements after `position`.
- *Expects*: `il.size() <= capacity() - size()`.
- *Requires*: `std::is_copy_constructible<value_type>`.
- *Complexity*: Linear in `size()` and `il.size()`.
- *Remarks*: If an exception is thrown by `value_type`'s copy constructor and `is_nothrow_move_constructible_v<value_type>` is `true` there are no effects. Otherwise, if an exception is thrown by `value_type`'s copy constructor the effects are *unspecified*.

```
template <class... Args>
constexpr iterator emplace(const_iterator position, Args&&... args);
```

- *Effects*: Inserts an element constructed from `args...` at `position` and invalidates all references to elements after `position`.
- *Expects*: `size() < capacity()`.
- *Requires*: `std::is_constructible<value_type, Args...>`.

- *Complexity*: Linear in `size()`.
- *Remarks*: If an exception is thrown by `value_type`'s constructor from `args...` and `is_nothrow_move_constructible_v<value_type>` is `true` there are no effects. Otherwise, if an exception is thrown by `value_type`'s constructor from `args...` the effects are *unspecified*.

```
template <class... Args>
constexpr reference emplace_back(Args&&... args);
```

- *Effects*: Inserts an element constructed from `args...` at the end.
- *Expects*: `size() < capacity()`.
- *Requires*: `std::is_constructible<value_type, Args...>`.
- *Complexity*: Constant.
- *Remarks*: If an exception is thrown by `value_type`'s constructor from `args...` there are no effects.

```
constexpr void push_back(const value_type& x);
```

- *Effects*: Inserts a copy of `x` at the end.
- *Expects*: `size() < capacity()`.
- *Requires*: `std::is_copy_constructible<value_type>`.
- *Complexity*: Constant.
- *Remarks*: If an exception is thrown by `value_type`'s copy constructor there are no effects.

```
constexpr void push_back(value_type&& x);
```

- *Effects*: Moves `x` to the end.
- *Expects*: `size() < capacity()`.
- *Requires*: `std::is_move_constructible<value_type>`.
- *Complexity*: Constant.
- *Remarks*: If an exception is thrown by `value_type`'s move constructor there are no effects.

```
constexpr void pop_back();
```

- *Effects*: Removes the last element of the container and destroys it.
- *Expects*: `!empty()`.
- *Complexity*: Constant.

```
constexpr iterator erase(const_iterator position);
```

- *Effects*: Removes the element at `position`, destroys it, and invalidates references to elements after `position`.
- *Expects*: `position` in range `[begin(), end()]`.
- *Complexity*: Linear in `size()`.
- *Remarks*: If an exception is thrown by `value_type`'s move constructor the effects are *unspecified*.

```
constexpr iterator erase(const_iterator first, const_iterator last);
```

- *Effects*: Removes the elements in range `[first, last)`, destroying them, and invalidating references to elements after `last`.
- *Expects*: `[first, last]` in range `[begin(), end()]`.
- *Complexity*: Linear in `size()` and `distance(first, last)`.
- *Remarks*: If an exception is thrown by `value_type`'s move constructor the effects are *unspecified*.

```
constexpr void swap(static_vector& x)
  noexcept(is_nothrow_swappable_v<value_type> &&
           is_nothrow_move_constructible_v<value_type>);
```

- *Effects*: Exchanges the contents of `*this` with `x`. All references to the elements of `*this` and `x` are invalidated.
- *Complexity*: Linear in `size()` and `x.size()`.

5.7 `static_vector` specialized algorithms

```
template <typename T, size_t N>
constexpr void swap(static_vector<T, N>& x,
                    static_vector<T, N>& y)
  noexcept(noexcept(x.swap(y)));
```

- *Constraints*: This function shall not participate in overload resolution unless `is_swappable_v<T>` is `true`.
- *Effects*: As if by `x.swap(y)`.
- *Complexity*: Linear in `size()` and `x.size()`.

6. Acknowledgments

The following people have significantly contributed to the development of this proposal. This proposal is based on Boost.Container's `boost::container::static_vector` and my extensive usage of this class over the years. As a consequence the authors of Boost.Container (Adam Wulkiewicz, Andrew Hundt, and Ion Gaztanaga) have had a very significant indirect impact on this proposal. The implementation of libc++ `std::vector` and the libc++ test-suite have been used extensively while prototyping this proposal, such that its author, Howard Hinnant, has had a significant indirect impact on the result of this proposal as well. The following people provided valuable feedback that influenced some aspects of this proposal: Walter Brown, Zach Laine, Rein Halbersma, and Andrzej Krzemieński. But I want to wholeheartedly acknowledge Casey Carter for taking the time to do a very detailed analysis of the whole proposal, which was invaluable and reshaped it in fundamental ways.

7. References

- [1] Boost.Container::static_vector: http://www.boost.org/doc/libs/1_59_0/doc/html/boost/container/static_vector.html .
- [2] EASTL fixed_vector: https://github.com/questor/eastl/blob/master/fixed_vector.h#L71 .
- [3] Folly small_vector: https://github.com/facebook/folly/blob/master/folly/docs/small_vector.md .
- [4] Howard Hinnant's stack_alloc: https://howardhinnant.github.io/stack_alloc.html .
- [5] P0494R0: `contiguous_container` proposal: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0494r0.pdf
- [6] P0597R0: `std::constexpr_vector<T>`: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0597r0.html
- [7] P0639R0: Changing attack vector of the `constexpr_vector`: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0639r0.html .
- [8] PR0274: Clump – A Vector-like Contiguous Sequence Container with Embedded Storage: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0274r0.pdf
- [9] Boost.Container::small_vector: http://www.boost.org/doc/libs/master/doc/html/boost/container/small_vector.html.
- [10] LLVM small_vector: http://llvm.org/docs/doxygen/html/classllvm_1_1SmallVector.html .
- [11] EASTL design: https://github.com/questor/eastl/blob/master/doc/EASTL%20Design.html#L284 .
- [12] Interest in StaticVector - fixed capacity vector: https://groups.google.com/d/topic/boost-developers-archive/4n1QuJyKTTk/discussion .
- [13] Stack-based vector container: https://groups.google.com/d/topic/boost-developers-archive/9BEXjV8ZMeQ/discussion.
- [14] static_vector: fixed capacity vector update: https://groups.google.com/d/topic/boost-developers-archive/d5_Kp-nmW6c/discussion.